5장. 토큰화

5.1 자연어 처리와 토큰

자연어와 자연어 처리

- 자연어는 사람들이 언어 활동을 위해 자연스럽게 만들어 낸 언어로, 인위적으로 만든 프로그래밍 언어와 구분된다.
- 자연어 처리(NLP)는 컴퓨터가 인간의 언어를 이해하고 해석 및 생성하기 위한 기술이다.
- 자연어 처리에서 해결해야 하는 문제는 다음과 같다.
  - 모호성(Ambiguity): 같은 단어나 구도 맥락에 따라 여러 의미를 가진다.
  - 가변성(Variability): 사투리, 강세, 신조어, 작문 스타일 등으로 언어가 매우 가변적이다.
  - 구조(Structure): 문장의 구문(Syntactic)을 파악해야 의미(Semantic)를 해석할 수 있다.

말뭉치, 토큰, 토큰화

- 말뭉치(Corpus)는 언어 모델을 훈련하고 평가하는 데 사용되는 대규모 자연어 텍스트 데이터다.
- 토큰(Token)은 개별 단어, 문장 부호 등 텍스트를 이루는 말뭉치보다 더 작은 단위다.
- 토큰화(Tokenization)는 텍스트를 컴퓨터가 이해할 수 있는 토큰으로 나누는 과정이다.
- 토크나이저(Tokenizer)는 텍스트 문자열을 토큰으로 나누는 알고리즘 또는 소프트웨어다.
- 토큰을 나누는 기준은 시스템이나 상황에 따라 다르며, 어떻게 나누느냐에 따라 시스템의 성능이 크게 달라지기도 한다.

토크나이저를 구축하는 방법

1. 공백 분할: 텍스트를 공백 단위로 분리해 개별 단어로 토큰화한다.
2. 정규표현식 적용: 정규 표현식으로 특정 패턴을 식별해 텍스트를 분할한다.
3. 어휘 사전(Vocabulary) 적용: 사전에 정의된 단어 집합을 토큰으로 사용한다.
4. 머신러닝 활용: 데이터세트를 기반으로 토큰화 방법을 학습한 머신러닝을 적용한다.

어휘 사전과 OOV

- 어휘 사전 방식은 직접 어휘 사전을 구축하므로 사전에 없는 단어나 토큰이 존재할 수 있다.
- 이렇게 사전에 없는 토큰을 OOV(Out of Vocab)라고 하며, OOV 문제를 고려해 토큰화하는 것이 중요하다.

- 큰 어휘 사전을 구축하면 학습 비용이 증가할 뿐 아니라 차원의 저주(Curse of Dimensionality)에 빠질 수 있다.
    - 모든 토큰을 벡터화하면 어휘 사전에 등장하는 토큰 개수만큼의 차원이 필요하다.
    - 벡터값이 거의 모두 0인 희소(sparse) 데이터가 되고, 이 표현은 출현 빈도만 고려해 토큰들의 순서 관계를 잘 표현하지 못한다.
    - 차원의 저주란 학습 데이터의 차원이 증가할수록 학습에 필요한 데이터가 증가하거나 모델의 성능이 저하되는 현상이다.

5.2 단어 및 글자 토큰화

단어 토큰화

- 단어 토큰화(Word Tokenization)는 텍스트 데이터를 의미 있는 단위인 단어로 분리하는 작업이다.
- 띄어쓰기, 문장 부호, 대소문자 등의 특정 구분자를 활용해 수행된다.
- 품사 태깅, 개체명 인식, 기계 번역 등에서 널리 사용되는 가장 일반적인 토큰화 방법이다.
- 문자열의 split 메서드를 이용하면 쉽게 수행할 수 있으며, 구분자를 입력하지 않으면 공백(Whitespace)을 기준으로 나눈다.
- 단점
    - 느낌표 하나의 차이로도 서로 다른 토큰이 된다. 예를 들어 '최고!'와 '최고!!'는 다른 토큰이다.
    - 'cg'와 'cg.'처럼 비슷한 의미도 서로 다른 토큰으로 취급되어, 사전에 'cg'가 있어도 'cg.', 'cg는', 'cg도' 등은 OOV가 된다.
    - 한국어 접사, 문장 부호, 오타, 띄어쓰기 오류에 취약하다.

글자 토큰화

- 글자 토큰화(Character Tokenization)는 띄어쓰기뿐 아니라 글자 단위로 문장을 나누는 방식이다.
- 장점
    - 비교적 작은 단어 사전을 구축할 수 있어 학습 시 컴퓨터 자원을 아낄 수 있다.
    - 전체 말뭉치를 학습할 때 각 단어를 더 자주 학습할 수 있다.
- 언어 모델링과 같은 시퀀스 예측 작업에서 유용하다.
- 영어는 글자 토큰화를 하면 각 알파벳으로 나뉘지만, 한글은 하나의 글자가 여러 자음과 모음의 조합으로 이루어져 있다.

자소 단위 토큰화

- 자소(字素)는 언어의 문자 체계에서 의미상 구별할 수 있는 가장 작은 단위다. 영어에서는 a, b, c, 한글에서는 ㄱ, ㄴ, ㅏ, ㅑ 등에 해당한다.

- 한글을 자소 단위로 나누는 토큰화에는 자모(jamo) 라이브러리를 활용한다.
- 자모 변환 함수는 완성형 한글을 유니코드 U+1100~U+11FE 사이의 조합형 한글 자모로 변환한다.
  - 컴퓨터가 한글을 인코딩하는 방식은 조합형과 완성형으로 나뉜다.
  - 조합형은 글자를 자모 단위로 나눠 인코딩한 뒤 이를 조합해 한글을 표현한다.
  - 완성형은 조합된 글자 자체에 값을 부여해 인코딩한다.
- 한글 호환성 자모 변환 함수는 조합형 한글을 초성, 중성, 종성으로 나눈 자소 단위로 반환한다.
- 장점
  - 단어 토큰화에 비해 훨씬 작은 크기의 단어 사전을 구축할 수 있다.
  - 단어 토큰화의 단점이었던 접사와 문장 부호의 의미를 학습할 수 있고, OOV를 획기적으로 줄일 수 있다.
- 단점
  - 개별 토큰은 아무런 의미가 없으므로 모델이 각 토큰의 의미를 조합해 결과를 도출해야 한다.
  - 개체명 인식(NER) 등에서 다의어나 동음이의어가 많은 도메인은 구별하기 어렵다.
  - 입력 시퀀스의 길이가 길어질수록 연산량이 증가한다. 교재 예시 문장은 단어 토큰화 13, 글자 토큰화 53, 자소 토큰화 101로 길어졌다.

5.3 형태소 토큰화

형태소와 형태소 토큰화

- 형태소 토큰화(Morpheme Tokenization)는 텍스트를 형태소 단위로 나누는 방법으로, 언어의 문법과 구조를 고려해 단어를 분리하고 의미 있는 단위로 분류한다.
- 한국어와 같은 교착어(Agglutinative Language)에서 중요하다. 한국어는 어근에 다양한 접사와 조사가 결합되어 하나의 낱말을 이루므로 각 형태소를 적절히 구분해야 한다.
- 형태소(Morpheme)는 실제로 의미를 가지고 있는 최소의 단위다.
  - 자립 형태소(Free Morpheme): 스스로 의미를 가지는 형태소로, 명사, 동사, 형용사 같은 단어를 이루는 기본 단위다.
  - 의존 형태소(Bound Morpheme): 스스로 의미를 갖지 못하고 다른 형태소와 조합되어 사용되며, 조사, 어미, 접두사, 접미사 등이 해당한다.
- 형태소 분석을 통해 문장 내 각 형태소의 역할을 파악하고, 이를 바탕으로 문장을 이해하고 처리할 수 있다.

형태소 어휘 사전

- 형태소 어휘 사전(Morpheme Vocabulary)은 단어의 집합인 어휘 사전 중에서도 각 단어의 형태소 정보를 포함하는 사전이다.

- 단어가 어떤 형태소들의 조합으로 이루어져 있는지에 대한 정보를 담고 있어 형태소 분석 작업에서 중요한 역할을 한다.
- 띄어쓰기 기준 어휘 사전에서는 '그는', '그녀는'을 같은 의미 단위로 인식하지 못하지만, 형태소 단위로 어휘 사전을 구축하면 '그녀'의 정보만 새로 학습해 '그녀는'이나 '그녀에게' 같은 어휘를 쉽게 학습할 수 있다.
- 일반적으로 형태소 어휘 사전에는 각 형태소가 속하는 품사와 해당 품사의 뜻 등의 정보도 함께 제공된다.
- 품사 태깅(Part Of Speech, POS Tagging)은 형태소 분석한 텍스트 데이터에서 각 형태소에 해당하는 품사를 태깅하는 작업이다. 이를 통해 문맥을 고려한 더욱 정확한 분석이 가능해진다.

KoNLPy

- 한국어 자연어 처리를 위해 개발된 라이브러리로, 명사 추출, 형태소 분석, 품사 태깅 등의 기능을 제공한다.
- 자바 기반의 한국어 형태소 분석기로, 자바 개발 키트(JDK) 기반으로 개발되어 JDK가 설치되어 있어야 한다.
- Okt(Open Korean Text), 꼬꼬마(Kkma), 코모란(Komoran), 한나눔(Hannanum), 메캅(Mecab) 등의 형태소 분석기를 지원한다. 메캅은 윈도우 환경에서 지원되지 않는다.
- Okt는 SNS 텍스트 데이터 기반으로 개발되었고, 꼬꼬마는 국립국어원의 세종 말뭉치를 기반으로 학습되었다.
- Okt의 대표 메서드
  - 명사 추출(okt.nouns)
  - 구문 추출(okt.phrases)
  - 형태소 추출(okt.morphs)
  - 품사 태깅(okt.pos): (형태소, 품사) 튜플의 리스트를 반환한다.
- 꼬꼬마는 Kkma 클래스를 통해 명사, 문장, 형태소, 품사를 추출한다. 구문 추출 대신 문장 추출(kkma.sentences) 기능을 제공한다.
- 같은 메서드라도 분석기의 특징에 따라 결과가 다르다.
  - 명사 추출에서 Okt는 '낼', '수'를 반환했지만 꼬꼬마는 반환하지 않았다.
  - 형태소 추출에서 꼬꼬마는 '할'을 '하', 'ㄹ'로 더 분리했다.
- 품사 태깅 체계도 다르다. Okt는 총 19개 품사를, 꼬꼬마는 더 세분화해 56개 품사를 사용한다.
  - 태깅하는 품사 수가 많으면 더 자세한 분석이 가능하지만 소요 시간이 길어진다.
  - 더 많은 품사로 분리하면 모델의 성능이 저하될 수도 있다.
  - 그러므로 시스템의 목적과 환경에 맞는 형태소 분석기를 사용해야 한다.

NLTK

- NLTK(Natural Language Toolkit)는 토큰화, 형태소 분석, 구문 분석, 개체명 인식, 감성 분석 등의 기능을 제공하는 자연어 처리 라이브러리다.

- 주로 영어 자연어 처리를 위해 개발되었지만 네덜란드어, 프랑스어, 독일어 등 다양한 언어의 데이터와 모델도 제공한다.
- 토큰화나 품사 태깅을 하려면 해당 패키지나 모델을 내려받아야 하며, Punkt 모델과 Averaged Perceptron Tagger 모델을 활용한다.
    - 두 모델 모두 트리뱅크(Treebank)라는 대규모의 영어 말뭉치를 기반으로 학습되었다.
    - Punkt 모델은 통계 기반 모델이며, Averaged Perceptron Tagger는 퍼셉트론 기반으로 품사 태깅을 수행한다.
- 단어 토크나이저(word_tokenize)는 문장을 공백 기준으로 단어로 분리하고, 구두점 등을 처리해 각각의 토큰을 리스트로 반환한다.
- 문장 토크나이저(sent_tokenize)는 마침표, 느낌표, 물음표 등의 구두점을 기준으로 문장을 분리한다.
- 영어는 한국어보다 토큰화하기 쉬운 구조를 가지고 있다.
- 품사 태깅(pos_tag)은 토큰화된 단어들을 입력으로 받아 품사를 태깅한다. Averaged Perceptron Tagger 모델은 총 35개의 품사를 태깅할 수 있다.

spaCy

- 사이썬(Cython) 기반으로 개발된 오픈 소스 자연어 처리 라이브러리다.
- NLTK와의 차이점
    - NLTK는 학습 목적으로 다양한 알고리즘과 예제를 제공한다.
    - spaCy는 빠른 속도와 높은 정확도를 목표로 하는 머신러닝 기반 라이브러리로, NLTK 모델보다 더 크고 복잡하며 더 많은 리소스를 요구한다.
- GPU 가속을 비롯해 영어, 프랑스어, 한국어, 일본어 등 24개 이상의 언어로 사전 학습된 모델을 제공한다.
- 사전 학습된 모델을 기반으로 처리하므로 모델 불러오기 함수(load)로 모델을 설정한다.
- 객체 지향적(Object Oriented)으로 구현되어, 처리 결과를 doc 객체에 저장한다. doc 객체는 여러 token 객체로 이루어져 있다.
- token 객체의 속성
    - 기본 품사 속성(pos_)
    - 세분화 품사 속성(tag_)
    - 원본 텍스트 데이터(text)
    - 토큰 사이의 공백을 포함하는 텍스트 데이터(text_with_ws)
    - 벡터(vector)
    - 벡터 노름(vector_norm)

형태소 분석과 품사 태깅의 의의와 한계

- 형태소 분석과 품사 태깅은 문장에서 의미를 갖는 최소 단위를 다루므로 자연어 처리에서 매우 중요한 전처리 작업이다.
- 띄어쓰기와 맞춤법이 잘 지켜지지 않는 경우나 신조어, 외래어, 특정 도메인의 축약어 등은 완벽히 대응하기 어렵다.
- 그럼에도 단어의 의미를 파악하고 문장의 구조를 이해하기 위해 필수적이며, 주요 키워드 추출이나 문장의 긍정/부정 판단 등 언어 모델의 성능을 높이는 데 중요한 역할을 한다.

5.4 하위 단어 토큰화

하위 단어 토큰화의 필요성

- 언어는 시간이 지남에 따라 변화하고 새로운 단어와 표현이 등장한다. 일상 언어에서는 맞춤법이나 띄어쓰기가 엄격하게 지켜지지 않고 신조어, 고유어가 빈번하게 생긴다.
- 디지털 매체에서 생성되는 텍스트는 오탈자가 발생할 확률이 높다.
- 기존 형태소 분석기는 전문용어나 고유어가 많은 데이터에서 약점을 보인다. 모르는 단어를 적절한 단위로 나누는 것에 취약하며, 잠재적으로 어휘 사전의 크기를 크게 만들고 OOV에 대응하기 어렵게 만든다.
    - 외래어 '시보리'를 인식하지 못해 '시', '보리'로 나눈다.
    - '어벙하다'라는 표현이 '어', '벙하지안구'로 나뉜다.
    - 신조어 '돈쭐내다'는 '돈쭐', '날', '만'으로 나뉘어, '돈쭐-'이라는 어근에 올 수 있는 모든 어미 조합이 하나의 토큰으로 인식되어 어휘 사전이 매우 커진다.
- 이를 해결하기 위한 방법 중 하나가 하위 단어 토큰화(Subword Tokenization)다.
- 하위 단어 토큰화는 하나의 단어를 빈번하게 사용되는 하위 단어(Subword)의 조합으로 나누어 토큰화하는 방법이다. 예를 들어 'Reinforcement'는 'Rein', 'force', 'ment' 등으로 나눌 수 있다.
- 장점
    - 단어의 길이를 줄여 처리 속도가 빨라진다.
    - OOV 문제, 신조어, 은어, 고유어 등으로 인한 문제를 완화할 수 있다.
- 하위 단어 토큰화 방법으로는 바이트 페어 인코딩, 워드피스, 유니그램 모델 등이 있다.

바이트 페어 인코딩(BPE)

- 바이트 페어 인코딩(Byte Pair Encoding, BPE)은 다이그램 코딩(Digram Coding)이라고도 하며, 하위 단어 토큰화의 한 종류다.
- 텍스트 데이터에서 가장 빈번하게 등장하는 글자 쌍의 조합을 찾아 부호화하는 압축 알고리즘으로, 초기에는 데이터 압축을 위해 개발되었으나 자연어 처리에서 하위 단어 토큰화 방법으로 사용된다.

- 알고리즘 동작
  1. 입력 데이터에서 가장 많이 등장한 글자 쌍의 빈도수를 측정하고, 가장 빈도수가 높은 글자 쌍을 탐색한다.
  2. 입력 데이터에 없는 새로운 글자로 치환한다. 치환되는 새로운 글자는 어떠한 글자를 사용해도 상관없다.
  3. 치환된 글자도 글자 쌍에 포함될 수 있으므로, 이 탐색과 부호화를 반복한다.
  4. 연속된 글자 쌍이 더 이상 나타나지 않거나 정해진 어휘 사전 크기에 도달하면 종료한다.
- 토크나이저로서의 BPE는 자주 등장하는 글자 쌍을 찾아 치환하는 대신 어휘 사전에 추가한다.
- 말뭉치에 적용하는 과정
  1. 빈도 사전과 어휘 사전을 만든다. 빈도 사전의 모든 단어를 글자 단위로 나눈다.
  2. 가장 자주 등장하는 글자 쌍을 병합하고 어휘 사전에 추가한다.
  3. 이를 정해진 횟수만큼 반복한다.
- 학습 후에는 말뭉치에 없던 단어(예: 'newer', 'wider', 'lowest')도 OOV로 처리되지 않고 기존 어휘 사전을 참고해 하위 단어로 토큰화할 수 있다.
- 대량의 말뭉치로 토크나이저를 학습할 때는 이미 구현된 토크나이저 라이브러리를 사용한다.

센텐스피스

- 센텐스피스(Sentencepiece)는 구글에서 개발한 오픈소스 하위 단어 토크나이저 라이브러리다.
  - 바이트 페어 인코딩과 유사한 알고리즘을 사용해 입력 데이터를 토큰화하고 단어 사전을 생성한다.
  - 워드피스, 유니코드 기반의 다양한 알고리즘을 지원하며, 사용자가 직접 설정할 수 있는 하이퍼파라미터를 제공한다.
- 코포라(Korpora)는 국립국어원이나 AI Hub에서 제공하는 말뭉치 데이터를 쉽게 사용할 수 있게 제공하는 오픈소스 라이브러리다.
- 토크나이저 모델 학습은 SentencePieceTrainer 클래스의 Train 메서드로 수행한다. 문자열 입력을 통해 인자들을 전달받는다.
- 주요 하이퍼파라미터
  - input: 말뭉치 텍스트 파일의 경로
  - model_prefix: 모델 파일 이름
  - vocab_size: 어휘 사전 크기
  - character_coverage: 말뭉치 내에 존재하는 글자 중 토크나이저가 다룰 수 있는 글자의 비율
  - model_type: 토크나이저 알고리즘(unigram, bpe, char, word)
  - max_sentence_length: 최대 문장 길이
  - unk_id: 어휘 사전에 없는 OOV를 의미하는 unk 토큰의 id (기본값 0)
  - bos_id: 문장이 시작되는 지점을 의미하는 bos 토큰의 id (기본값 1)

    - eos_id: 문장이 끝나는 지점을 의미하는 eos 토큰의 id (기본값 2)
- 학습이 완료되면 model 파일(학습된 토크나이저)과 vocab 파일(어휘 사전)이 생성된다.
- 센텐스피스는 띄어쓰기나 공백도 특수문자로 취급해 '_'(U+2581)로 공백을 표현한다.
- SentencePieceProcessor 클래스로 학습된 모델을 불러와 사용한다.
    - encode_as_pieces: 문장을 토큰화한다.
    - encode_as_ids: 토큰을 정수로 인코딩한다. 이 정수는 어휘 사전의 토큰에 매핑된 ID 값이다.
    - decode_ids, decode_pieces: 정수나 하위 단어 토큰을 문자열로 되돌린다.
    - get_piece_size: 하위 단어의 개수를 반환한다.
    - id_to_piece: 정숫값을 하위 단어로 변환한다.
- 어휘 사전에는 특수 토큰이 존재한다. unk는 unknown의 약자로 OOV 발생 시 매핑되는 토큰이며, s와 /s는 문장의 시작 지점과 종료 지점을 표시하는 토큰이다.

워드피스

- 워드피스(Wordpiece)는 바이트 페어 인코딩과 유사한 방법으로 학습되지만, 빈도 기반이 아닌 확률 기반으로 글자 쌍을 병합한다.
- 모델이 새로운 하위 단어를 생성할 때 이전 하위 단어와 함께 나타날 확률을 계산해 가장 높은 확률을 가진 하위 단어를 선택한다. 이렇게 선택된 하위 단어는 이후에 더 높은 확률로 선택될 가능성이 높다.
- 글자 쌍 병합 점수는 score = f(x, y) / (f(x) × f(y))로 계산한다.
    - f는 빈도(frequency)를 나타내는 함수이고, x와 y는 병합하려는 하위 단어다.
    - f(x, y)는 x와 y가 조합된 글자 쌍의 빈도다.
    - 점수는 x와 y를 병합하는 것이 적절한지 판단하기 위한 점수다.
- 워드피스 토크나이저도 BPE와 마찬가지로 연속된 글자 쌍이 더 이상 나타나지 않거나 정해진 어휘 사전 크기에 도달할 때까지 학습한다.

토크나이저스(Tokenizers) 라이브러리

- 허깅 페이스의 토크나이저스 라이브러리의 워드피스 API를 이용하면 토크나이저를 쉽고 빠르게 구현하고 학습할 수 있다.
- 정규화(Normalization)와 사전 토큰화(Pre-tokenization)를 제공한다.
    - 정규화: 일관된 형식으로 텍스트를 표준화하고 모호한 경우를 방지하기 위해 일부 문자를 대체하거나 제거한다. 불필요한 공백 제거, 대소문자 변환, 유니코드 정규화, 구두점 처리, 특수 문자 처리 등이 있다.
    - 사전 토큰화: 입력 문장을 토큰화하기 전에 단어와 같은 작은 단위로 나눈다. 공백이나 구두점 기준으로 나눠 효율적으로 처리하고 모델의 성능을 향상시킬 수 있다.

- Tokenizer 객체에 WordPiece 모델을 불러온 뒤 정규화 방식(normalizers 모듈, Sequence 형식)과 사전 토큰화 방식(pre_tokenizers 모듈)을 설정하고, train 메서드로 학습하고 save 메서드로 저장한다.
- 정규화 클래스
  - NFD, NFKD, NFC, NFKC: 각각의 유니코드 정규화
  - Lowercase: 입력 문장의 모든 대문자를 소문자로 변환
  - Strip: 입력 문장의 양 끝에 있는 공백 제거
  - StripAccents: 모든 엑센트 기호 제거
  - Replace: 문자 혹은 정규 표현식을 기반으로 입력 문장 변환
  - BertNormalizer: BERT 모델에 사용된 정규화 적용
  - Sequence: 여러 개의 정규화 모듈을 병합해 순서대로 수행
- 사전 토큰화 클래스
  - Sequence: 여러 개의 사전 토큰화 모듈을 병합해 순서대로 수행
  - ByteLevel: OpenAI에서 GPT-2를 학습할 때 사용한 방법으로 문자열을 그래픽 문자열로 매핑하고 공백을 기준으로 나눈다.
  - CharDelimiterSplit: 문자 기준으로 문자열을 나눈다.
  - Digits: 모든 형태의 숫자 표현을 기준으로 문자열을 나눈다.
  - Punctuation: 구두점을 기준으로 입력 문자열을 나눈다.
  - Metaspace: Whitespace 기준으로 사전 토큰화를 진행하고 replacement로 Whitespace를 치환한다. 기본값은 _(U+2581)이다.
  - Whitespace: 단어 경계(공백과 구두점)를 기준으로 사전 토큰화를 진행한다. \w+|[^\w\s]+ 정규 표현식을 적용한다.
  - WhitespaceSplit: 공백 문자를 기준으로 입력 문자열을 나눈다.
  - Split: 정규표현식(pattern)과 동작(behavior)을 기준으로 문자열을 나눈다. behavior에는 removed(삭제), isolated(개별 토큰 처리), merged_with_previous(이전 문자열과 결합), merged_with_next(다음 문자열과 결합), contiguous(다음 문자열이 토큰화되지 않아도 결합)가 있다.
- 학습이 끝나면 JSON 파일이 생성되며, 정규화 및 사전 토큰화 등의 메타데이터와 함께 어휘 사전이 저장된다.
- 워드피스 토큰화는 저장된 JSON으로 Tokenizer 객체를 만든 뒤 디코더를 WordPieceDecoder로 설정해 수행한다.
  - encode: 문장을 토큰화하고 각 토큰의 색인 번호를 반환한다.
  - encode_batch: 여러 문장을 한 번에 토큰화한다.
  - 토큰화된 데이터는 토큰(tokens) 속성으로, 정수 인코딩된 ID는 정수(ids) 속성으로 확인한다.
  - decode: 정수 인코딩된 결과를 다시 문장으로 디코딩한다.

하위 단어 토큰화 정리

- 최근 연구 동향은 더 큰 말뭉치를 사용해 모델을 학습하고 OOV의 위험을 줄이기 위해 하위 단어 토큰화를 활용하는 것이다.
- 이 외에도 유니코드 단위가 아닌 바이트 단위에서 토큰화하는 바이트 수준 바이트 페어 인코딩(Byte-level Byte-Pair-Encoding, BBPE)이나, 크기가 큰 어휘 사전에서 덜 필요한 토큰을 제거하며 학습하는 유니그램(Unigram) 등이 있다.

6장. 임베딩

6.1 텍스트 벡터화와 임베딩

텍스트 벡터화

- 토큰화만으로는 모델을 학습할 수 없다. 컴퓨터는 텍스트 자체를 이해할 수 없으므로 텍스트를 숫자로 변환하는 텍스트 벡터화(Text Vectorization) 과정이 필요하다.
- 기초적인 텍스트 벡터화에는 원-핫 인코딩(One-Hot Encoding)과 빈도 벡터화(Count Vectorization) 등이 있다.

원-핫 인코딩과 빈도 벡터화

- 원-핫 인코딩: 문서에 등장하는 각 단어를 고유한 색인 값으로 매핑한 후, 해당 색인 위치를 1로 표시하고 나머지 위치는 모두 0으로 표시하는 방식이다.
- 빈도 벡터화: 문서에서 단어의 빈도수를 세어 해당 단어의 빈도를 벡터로 표현하는 방식이다. 원-핫 인코딩은 같은 단어가 여러 번 나와도 1과 0으로 표현하지만, 빈도 벡터화는 해당 단어의 빈도를 표시한다.
- 장점: 단어나 문장을 벡터 형태로 변환하기 쉽고 간단하다.
- 단점
  - 벡터의 희소성(Sparsity)이 크다. 말뭉치 내에 존재하는 토큰의 개수만큼의 차원을 가져야 하는데 문장 내 토큰 수는 현저히 적어, 컴퓨팅 비용의 증가와 차원의 저주 문제가 생긴다.
  - 텍스트의 벡터가 입력 텍스트의 의미를 내포하지 않아, 두 문장이 의미적으로 유사해도 벡터가 유사하게 나타나지 않을 수 있다. 예를 들어 'i scream for ice cream'과 'ice cream for i scream'은 두 방식 모두 같은 벡터로 변환된다.

워드 임베딩과 동적 임베딩

- 이러한 문제를 해결하기 위해 워드 투 벡터(Word2Vec)나 패스트텍스트(fastText) 등과 같이 단어의 의미를 학습해 표현하는 워드 임베딩(Word Embedding) 기법을 사용한다.

- 워드 임베딩은 단어를 고정된 길이의 실수 벡터로 표현하는 방법으로, 단어의 의미를 벡터 공간에서 다른 단어와의 상대적 위치로 표현해 단어 간의 관계를 추론한다.
- 워드 임베딩은 고정된 임베딩을 학습하기 때문에 다의어나 문맥 정보를 다루기 어렵다는 단점이 있어, 인공신경망을 활용해 동적 임베딩(Dynamic Embedding) 기법을 사용한다.

6.2 언어 모델

언어 모델

- 언어 모델(Language Model)은 입력된 문장으로 각 문장을 생성할 수 있는 확률을 계산하는 모델이다. 주어진 문장을 바탕으로 문맥을 이해하고 문장 구성에 대한 예측을 수행한다.
- 자동 번역, 음성 인식, 텍스트 요약 등 다양한 자연어 처리 분야에서 활용된다.
- 어떤 문장 뒤에 올 수 있는 문장은 사실상 무한하므로, 완성된 문장 단위로 확률을 계산하는 것은 불가능하다. 이를 해결하기 위해 문장 전체 대신 하나의 토큰 단위로 예측하는 자기회귀 언어 모델이 고안되었다.

자기회귀 언어 모델

- 자기회귀 언어 모델(Autoregressive Language Model)은 입력된 문장들의 조건부 확률을 이용해 다음에 올 단어를 예측한다. 조건부 확률은 이전 단어들의 시퀀스가 주어졌을 때 다음 단어의 확률을 계산하는 것을 의미한다.
- 이전에 등장한 모든 토큰의 정보를 고려해 문맥 정보를 파악하여 다음 단어를 생성한다. 다음 단어는 다시 이전 단어를 기반으로 예측이 이루어지며 이 과정이 반복된다.
- 언어 모델의 조건부 확률은 $P(w_t \mid w_1, w_2, ..., w_{t-1}) = P(w_1, ..., w_t) / P(w_1, ..., w_{t-1})$로 표현한다.
- 조건부 확률의 연쇄법칙(Chain rule for conditional probability)을 적용하면 $P(w_1, ..., w_t) = P(w_1) P(w_2 \mid w_1) ... P(w_t \mid w_1, ..., w_{t-1})$이 된다.
    - 문장 전체의 확률은 첫 번째 단어의 확률 $P(w_1)$과 각 단어가 이전 단어들의 조건부 확률에 따라 발생할 확률의 곱으로 나타낼 수 있다.
    - 이를 통해 언어 모델은 문장 전체의 확률을 계산하고, 그것을 이용해 다음 단어를 예측한다.
- 모델의 출력값이 다시 모델의 입력값으로 사용되는 특징 때문에 자기회귀라는 이름이 붙었다.
- 시점별로 다음에 올 토큰을 예측하는 것이므로 토큰 분류 문제로 정의할 수 있다.

통계적 언어 모델

- 통계적 언어 모델(Statistical Language Model)은 언어의 통계적 구조를 이용해 문장이나 단어의 시퀀스를 생성하거나 분석한다. 시퀀스에 대한 확률 분포를 추정해 문장의 문맥을 파악하여 다음에 등장할 단어의 확률을 예측한다.

- 일반적으로 마르코프 체인(Markov Chain)을 이용해 구현된다. 마르코프 체인은 빈도 기반의 조건부 확률 모델 중 하나로, 이전 상태와 현재 상태 간의 전이 확률을 이용해 다음 상태를 예측한다.
- 빈도 기반 조건부 확률: 주어진 데이터에서 각 변수가 발생한 빈도수를 기반으로 확률을 계산한다. 예를 들어 '안녕하세요'가 1,000번, '안녕하세요 만나서'가 700번 등장했다면 P(만나서 | 안녕하세요) = 700 / 1000이다.
- 단점
  - 단어의 순서와 빈도에만 기초해 확률을 예측하므로 문맥을 제대로 파악하지 못하면 불완전하거나 부적절한 결과를 생성할 수 있다.
  - 한 번도 등장한 적이 없는 단어나 문장에 대해서는 정확한 확률을 예측하기 어렵다. 이렇게 관측한 적이 없는 데이터를 예측하지 못하는 문제를 데이터 희소성(Data sparsity) 문제라고 한다.
- 통계적 언어 모델은 기존에 학습한 텍스트 데이터에서 패턴을 찾아 확률 분포를 생성하므로, 이를 이용해 새로운 문장을 생성할 수 있으며 다양한 종류의 텍스트 데이터를 학습할 수 있다.
- 딥러닝 등의 인공지능 기술이 발전하면서 더욱 강력한 모델을 구현할 수 있게 되었다. 최근에는 언어 모델을 활용해 가중치를 사전 학습한다.
  - GPT(Generative Pre-trained Transformer): 이전 토큰을 입력으로 다음 토큰을 예측하는 방식으로 사전 학습한다. 인과적 언어 모델이다.
  - BERT(Bidirectional Encoder Representations from Transformers): 문장 중간의 토큰을 [MASK]로 가리고 이를 예측하는 방식으로 사전 학습한다. 마스킹된 언어 모델이다.

N-gram

- 가장 기초적인 통계적 언어 모델로, 텍스트에서 N개의 연속된 단어 시퀀스를 하나의 단위로 취급하여 특정 단어 시퀀스가 등장할 확률을 추정한다.
- 입력 텍스트를 하나의 토큰 단위로 분석하지 않고 N개의 토큰을 묶어서 분석한다.
  - N이 1일 때는 유니그램(Unigram), 2일 때는 바이그램(Bigram), 3일 때는 트라이그램(Trigram)이라 부른다.
  - N이 4 이상이면 N-gram이라고 부른다.
- 모든 토큰을 사용하지 않고 N-1개의 토큰만 고려해 확률을 계산한다. t번째 토큰의 조건부 확률은 P(w_t | w_{t-1}, w_{t-2}, ..., w_{t-N+1})이다.
  - 이전 단어들의 개수를 결정하는 N의 값을 조정해 N-gram 모델의 성능을 조절할 수 있다.
- 간단한 파이썬 코드 또는 NLTK 라이브러리로 구현할 수 있다.
- 통계적 언어 모델은 상대적으로 구현이 쉽다.
- 작은 규모의 데이터세트에서 연속된 문자열 패턴을 분석하는 데 큰 효과를 보이며, 관용적 표현 분석에도 활용된다. 예를 들어 '입이 무겁다'처럼 자주 등장하는 연속된 단어나 구를 추출하고 분석해 관용적 표현을 파악할 수 있다.

- 시퀀스에서 연속된 n개의 단어를 추출하므로 단어의 순서가 중요한 자연어 처리 작업 및 문자열 패턴 분석에 활용된다.

6.3 TF-IDF

BoW와 TF-IDF

- TF-IDF(Term Frequency-Inverse Document Frequency)는 텍스트 문서에서 특정 단어의 중요도를 계산하는 방법으로, 문서 내에서 단어의 중요도를 평가하는 데 사용되는 통계적인 가중치를 의미한다. 즉, BoW(Bag-of-Words)에 가중치를 부여하는 방법이다.
- BoW는 문서나 문장을 단어의 집합으로 표현하는 방법으로, 단어의 중복을 허용해 빈도를 기록한다.
  - 원-핫 인코딩은 단어의 등장 여부를 판별해 0과 1로 표현하지만, BoW는 등장 빈도를 고려해 표현한다.
- BoW의 한계
  - 모든 단어가 동일한 가중치를 갖는다. 영화 리뷰의 긍정/부정 분류 모델을 만든다면 높은 성능을 얻기 어렵다.
  - 'don't'처럼 문장 분류에 결정적인 역할을 하는 단어가 자주 등장하지 않기 때문에, 토큰화나 분류 모델 진행 시 해당 데이터가 무시될 수 있다.

단어 빈도(TF)

- 단어 빈도(Term Frequency, TF)는 문서 내에서 특정 단어의 빈도수를 나타내는 값이다. TF(t, d) = count(t, d)로 표현한다.
- BoW 벡터 표현 방법처럼 문서 내에서 단어가 등장한 빈도수를 계산하며, 해당 단어의 상대적인 중요도를 측정하는 데 사용된다.
- TF 값이 높을수록 특정 문서에서 중요한 역할을 한다고 생각할 수도 있지만, 단어 자체가 특정 문서 내에서 자주 사용되는 단어이므로 전문 용어나 관용어로 간주할 수 있다.
- TF는 단순히 단어의 등장 빈도수를 계산하기 때문에 문서의 길이가 길어질수록 해당 단어의 TF 값도 높아질 수 있다.

문서 빈도(DF)

- 문서 빈도(Document Frequency, DF)는 한 단어가 얼마나 많은 문서에 나타나는지를 의미한다. DF(t, D) = count(t ∈ d : d ∈ D)로 표현한다.
- DF는 단어가 몇 개의 문서에서 등장하는지 계산한다. 즉 3개의 문서에서 단어가 4번 등장하더라도 DF 값은 문서 수인 3이 된다.
- DF 값이 높으면 그 단어는 일반적으로 널리 사용되며 중요도가 낮을 수 있다.
- DF 값이 낮으면 특정 단어가 적은 수의 문서에만 등장한다는 뜻이므로, 특정한 문맥에서만 사용되는 단어일 가능성이 있으며 중요도가 높을 수 있다.

역문서 빈도(IDF)

- 역문서 빈도(Inverse Document Frequency, IDF)는 전체 문서 수를 문서 빈도로 나눈 다음에 로그를 취한 값이다. 문서 내에서 특정 단어가 얼마나 중요한지를 나타낸다.
- 문서 빈도가 높을수록 해당 단어가 일반적이고 상대적으로 중요하지 않다는 의미이므로, 문서 빈도의 역수를 취하면 단어의 빈도수가 적을수록 IDF 값이 커지게 보정하는 역할을 한다.
- IDF(t, D) = log(count(D) / (1 + DF(t, D)))
  - 분모에 DF에 1을 더하는 이유: 특정 단어가 한 번도 등장하지 않는다면 분모가 0이 되는 경우가 발생하므로, 1과 같은 작은 값을 더해 분모가 0이 되는 결과를 방지한다.
  - 로그를 취하는 이유: 전체 문서 수를 문서 빈도로 나눈 값을 그대로 쓰면 너무 큰 값이 나올 수 있다. 예를 들어 10,000개의 문서에서 특정한 단어가 1번 등장하면 IDF 값은 5,000이 된다. 이를 방지하고 정교한 가중치를 얻기 위해 로그를 취한다.

TF-IDF

- TF-IDF는 단어 빈도와 역문서 빈도를 곱한 값이다. TF-IDF(t, d, D) = TF(t, d) × IDF(t, D)
- 문서 내에 단어가 자주 등장하지만 전체 문서 내에 해당 단어가 적게 등장한다면 TF-IDF 값은 커진다. 그러므로 전체 문서에서 자주 등장할 확률이 높은 관사나 관용어 등의 가중치는 낮아진다.

사이킷런의 TfidfVectorizer

- 사이킷런(Scikit-learn)은 파이썬의 대표적인 머신러닝 라이브러리로, 전처리, 모델 파이프라인, 모델 검증 등에 필요한 전반적인 모듈을 제공한다. 말뭉치를 쉽게 TF-IDF 형태로 변환할 수 있는 TF-IDF 클래스를 제공한다.
- TfidfVectorizer의 주요 매개변수
  - 입력값(input): 입력될 데이터의 형태를 의미한다. 기본값 content는 문자열 데이터 혹은 바이트 형태의 입력값을 의미하며, 파일 객체를 사용한다면 file, 파일 경로를 사용하는 경우 filename으로 입력한다.
  - 인코딩(encoding): 바이트 혹은 파일을 입력값으로 받을 경우 사용할 텍스트 인코딩 값이다.
  - 소문자 변환(lowercase): 입력받은 데이터를 소문자로 변환할지 여부다. 참값으로 설정하면 모든 입력 텍스트를 소문자로 변환한다.
  - 불용어(stop_words): 분석에 도움이 되지 않는 의미 없는 단어들이다. 입력받은 단어들은 단어 사전에 추가되지 않는다.
  - N-gram 범위(ngram_range): 사용할 N-gram의 범위를 (최솟값, 최댓값) 형태로 입력한다. (1, 1)은 유니그램, (1, 2)는 유니그램과 바이그램을 사용한다는 의미다.
  - 최댓값 문서 빈도(max_df): 전체 문서 중 일정 횟수 이상 등장한 단어는 불용어로 처리한다. 정수를 입력하면 해당 등장 횟수를 초과해 등장하는 단어를 불용어 처리하고, 1 이하의 실수를 입력하면 해당 비율을 초과해 등장한 단어를 불용어 처리한다.

    - 최솟값 문서 빈도(min_df): 전체 문서 중 일정 횟수 미만으로 등장한 단어를 불용어로 처리한다는 뜻으로, 최댓값 문서 빈도와 동일한 패턴으로 입력할 수 있다.
    - 단어사전(vocabulary): 미리 구축한 단어 사전이 있다면 해당 단어 사전을 사용한다. 입력하지 않으면 TF-IDF 학습 시 자동으로 구축된다.
    - IDF 분모처리(smooth_idf): IDF 계산 시 분모에 1을 더한다.
- TfidfVectorizer 클래스는 fit 메서드를 통해 학습해야 데이터의 변환 또는 추론이 가능하다.
    - 객체의 학습이 완료되면 transform 메서드로 데이터 변환을 수행할 수 있다. 또는 fit_transform 메서드로 학습과 변환을 동시에 수행할 수 있다.
    - toarray 메서드는 TF-IDF를 넘파이 배열로 변환한다. (문서 수) × (단어 수)의 형태이며, 각 행은 하나의 문서, 각 열은 단어를 의미한다.
    - vocabulary_ 속성은 TF-IDF에 사용된 단어 사전을 의미한다. 키는 고유한 단어, 값은 해당 단어에 대한 색인 값이다.
- 점수가 가장 높은 값을 추려 단어 사전과 매핑하면 문서마다 중요한 단어만 추출할 수 있고, 벡터값을 활용해 문서 내 핵심 단어를 추출할 수 있다.

TF-IDF의 한계

- 빈도 기반 벡터화는 문장의 순서나 문맥을 고려하지 않는다. 그러므로 문장 생성과 같이 순서가 중요한 작업에는 부적합하다.
- 벡터가 해당 문서 내의 중요도를 의미할 뿐, 벡터가 단어의 의미를 담고 있지는 않다.
- 예를 들어 '나는 바나나를 먹었다', '그녀가 과일을 섭취한다', '고양이는 야옹 운다'를 벡터화하면, 앞의 두 문장이 의미가 더 가깝지만 세 문장 간의 유사도는 동일하다.